
Lab 05

Objective:

The objective of this lab is to learn how to implement learning vector quantization in python.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement LVQ model in python.
- Implement a training algorithm for LVQ in python

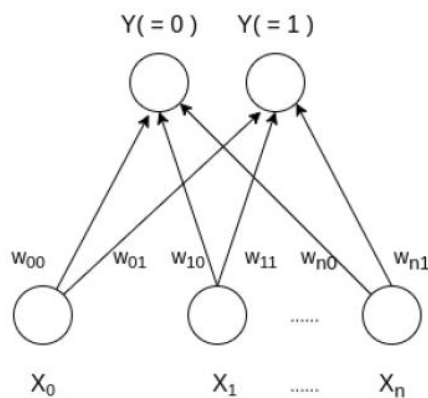
Instructor Note:

As pre-lab activity, read and implement the following tutorials:

1. <https://medium.com/@matteo.arellano/learning-vector-quantization-lvq-from-scratch-python-c7a5abcd79bd>
2. <https://machinelearningmastery.com/implement-learning-vector-quantization-scratch-python/>

11) Useful Concepts

Learning Vector Quantization (or LVQ) is a type of Artificial Neural Network which also inspired by biological models of neural systems. It is based on prototype supervised learning classification algorithm and trained its network through a competitive learning algorithm similar to Self Organizing Map. It can also deal with the multiclass classification problem. LVQ has two layers, one is the Input layer and the other one is the Output layer. The architecture of the Learning Vector Quantization with the number of classes in an input data and n number of input features for any sample is given below:



How Learning Vector Quantization works?: Let's say that an input data of size (m, n) where m is the number of training examples and n is the number of features in each example and a label vector of size $(m, 1)$. First, it initializes the weights of size (n, c) from the first c number of training samples with different labels and should be discarded from all training samples. Here, c is the number of classes. Then iterate over the remaining input data, for each training example, it updates the winning vector (weight vector with the shortest distance (e.g Euclidean distance) from the training example).

The weight updation rule is given by:

```
if correctly_classified:
     $w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha(t) * (x_i^k - w_{ij}(\text{old}))$ 
else:
     $w_{ij}(\text{new}) = w_{ij}(\text{old}) - \alpha(t) * (x_{ik} - w_{ij}(\text{old}))$ 
```

where α is a learning rate at time t , j denotes the winning vector, i denotes the i th feature of training example and k denotes the k th training example from the input data. After training the

LVQ network, trained weights are used for classifying new examples. A new example is labelled with the class of the winning vector.

Algorithm:

Step 1: Initialize reference vectors.

from a given set of training vectors, take the first "n" number of clusters training vectors and use them as weight vectors, the remaining vectors can be used for training.

Assign initial weights and classifications randomly

Step 2: Calculate Euclidean distance for $i=1$ to n and $j=1$ to m ,

$$D(j) = \sum (x_i - W_{ij})^2$$

find winning unit index j , where $D(j)$ is minimum

Step 3: Update weights on the winning unit w_i using the following conditions:

if $T = J$ then $w_i(\text{new}) = w_i(\text{old}) + \alpha[x - w_i(\text{old})]$

if $T \neq J$ then $w_i(\text{new}) = w_i(\text{old}) - \alpha[x - w_i(\text{old})]$

Step 4: Check for the stopping condition if false repeat the above steps.

12) Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>45</i>	<i>Medium</i>	<i>CLO-3</i>

Activity 1: Implementation of LVQs in python

Solution:

```
import math

class LVQ :

    # Function here computes the winning vector

    # by Euclidean distance

    def winner( self, weights, sample ) :

        D0 = 0

        D1 = 0

        for i in range( len( sample ) ) :
```

```

        D0 = D0 + math.pow( ( sample[i] - weights[0][i] ), 2 )

        D1 = D1 + math.pow( ( sample[i] - weights[1][i] ), 2 )

    if D0 > D1 :

        return 0

    else :

        return 1

# Function here updates the winning vector

def update( self, weights, sample, J, alpha, actual ) :

    if actual == j:

        for i in range(len(weights)) :

            weights[J][i] = weights[J][i] + alpha * ( sample[i] - weights[J][i] )

    else:

        for i in range(len(weights)) :

            weights[J][i] = weights[J][i] - alpha * ( sample[i] - weights[J][i] )

# Driver code

def main() :

    # Training Samples ( m, n ) with their class vector

```

```
X = [[ 0, 0, 1, 1 ], [ 1, 0, 0, 0 ],  
      [ 0, 0, 0, 1 ], [ 0, 1, 1, 0 ],  
      [ 1, 1, 0, 0 ], [ 1, 1, 1, 0 ],]  
  
Y = [ 0, 1, 0, 1, 1, 1 ]  
  
m, n = len( X ), len( X[0] )  
  
# weight initialization ( n, c )  
  
weights = []  
  
weights.append( X.pop( 0 ) )  
  
weights.append( X.pop( 0 ) )  
  
  
# Samples used in weight initialization will  
  
# not use in training  
  
m = m - 2  
  
Y.pop(0)  
  
Y.pop(0)  
  
  
# training  
  
ob = LVQ()
```

```
epochs = 3

alpha = 0.1

for i in range( epochs ) :

    for j in range( m ) :

        # Sample selection

        T = X[j]

        # Compute winner

        J = ob.winner( weights, T )

        # Update weights

        ob.update( weights, T, J, alpha , Y[j])

# classify new input sample

T = [ 0, 0, 1, 0 ]

J = ob.winner( weights, T )

print( "Sample T belongs to class : ", J )

print( "Trained weights : ", weights )

if __name__ == "__main__":
```

`main()`

13) Graded Lab Tasks

The purpose of this lab task is to help students understand and implement LVQs in python.

Lab Task 1: Build and Train LVQs in Python

Download a dataset from Kaggle or UCI machine learning repository and train SOMs algorithm on the dataset.

Instructions for Submission:

4. Submit a Jupyter Notebook or Python script with the complete implementation.
5. Include comments in the code to explain each step.
6. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics